System Architecture Documentation

End-to-End Enterprise RAG Pipeline: User to Response

Hooria Zahoor | BS Artificial Intelligence, The University of Faisalabad | AI Summer Internship 2026

This document describes the end-to-end architecture of an enterprise Retrieval-Augmented Generation (RAG) system, tracing a single request from the user through the frontend, backend, document processing pipeline, and vector search, to LLM-generated response. Steps 4–7 (Document Processing through Vector Database) run once, offline, whenever new documents are added; steps 1–3 and 8–10 run on every user query in real time. Figure 1 shows the full flow; each component is explained in detail with a concrete example below.

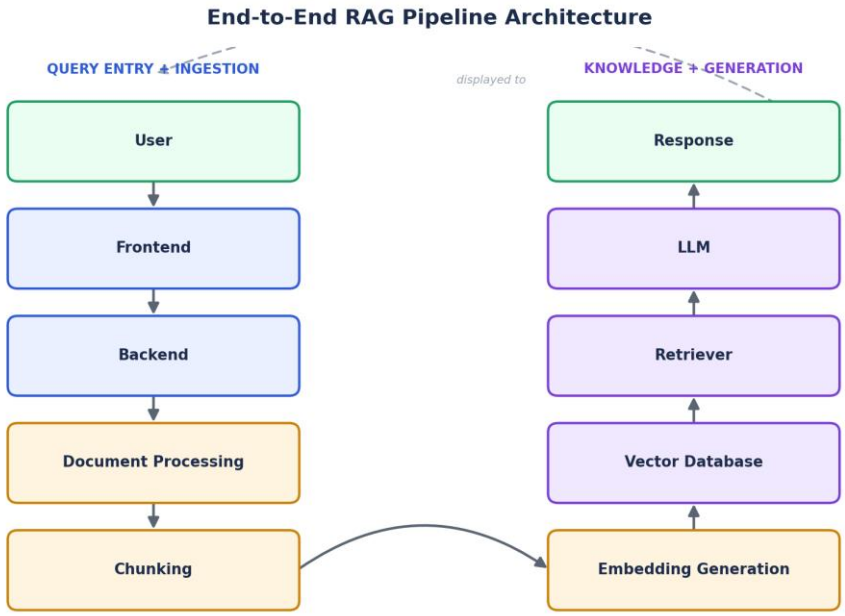

Figure 1. End-to-end pipeline: User → Frontend → Backend → Document Processing → Chunking → Embedding Generation → Vector Database → Retriever → LLM → Response.

Component Descriptions

1. User

The human initiating and consuming the interaction — an employee, customer, or internal team member who has a question that requires information from enterprise documents. The user is both the entry point (asks the question) and exit point (reads the answer) of the pipeline.

Example: *An HR employee opens the company chatbot and types, "How many days of maternity leave am I entitled to?"*

2. Frontend

The user-facing interface that captures input, renders the conversation, and displays the final answer with its sources. It manages the visual chat experience and session state but contains no business logic itself — it simply forwards requests to the backend and renders what comes back.

Example: *A Streamlit web app with a chat input box, a "Send" button, and an expandable "Sources" panel showing which document and page an answer came from.*

3. Backend

The server-side application that receives requests from the frontend and orchestrates the rest of the pipeline: calling the retriever, assembling the prompt, invoking the LLM, and packaging the response. It is the control layer that ties every other component together and enforces business rules such as authentication and access control.

Example: *A Python backend function `handle_query(question)` that calls `retrieve(question)`, builds a prompt from the results, calls the Gemini API, and returns the formatted answer.*

4. Document Processing

Ingests raw enterprise files — PDF, DOCX, TXT, Markdown — and extracts clean, usable text from them, stripping formatting noise, headers/footers, and encoding artifacts. This step runs offline whenever new or updated documents are added to the knowledge base, not on every query.

Example: *An uploaded IT_Security_Policy.pdf is parsed page-by-page with pypdf; extra whitespace and broken line breaks are cleaned before the text moves to chunking.*

5. Chunking

Splits the cleaned document text into smaller, overlapping segments sized to fit an embedding model's input window while preserving enough context to be meaningful on their own. Each chunk is tagged with metadata — source document, page number, chunk index — used later for citations.

Example: *A 3-page policy document is split into ~800-character chunks with 150-character overlap, producing 11 chunks, each labelled with its page number.*

6. Embedding Generation

Converts each text chunk into a dense numerical vector that captures its semantic meaning, so that chunks with similar meaning end up with mathematically similar vectors — regardless of exact wording. A sentence-embedding model performs this conversion once per chunk during ingestion.

Example: *The chunk "Employees receive 90 calendar days of paid maternity leave" is encoded by all-MiniLM-L6-v2 into a 384-dimension vector.*

7. Vector Database

Stores every chunk's embedding alongside its text and metadata, and provides fast nearest-neighbour search over potentially millions of vectors. It persists this index so that previously processed documents remain searchable across sessions without being re-embedded.

Example: *ChromaDB stores 5,000 chunks from 40 policy documents in a persistent collection on disk, ready for instant querying after the app restarts.*

8. Retriever

At query time, embeds the user's question with the same embedding model used during ingestion, then searches the vector database for the top-k chunks whose vectors are most similar to the query — typically using cosine similarity, optionally combined with keyword search or metadata filters.

Example: *The query "maternity leave days" is embedded and matched against the vector database, returning the 4 most semantically similar chunks, including the relevant HR policy excerpt.*

9. LLM

Receives a constructed prompt containing the retrieved chunks, the user's question, and recent conversation history, and generates a natural-language answer grounded strictly in that context — rather than relying on its own possibly outdated internal knowledge. It is instructed to cite its sources and to decline when the answer isn't present in the retrieved context.

Example: *Given the retrieved HR policy chunk, the Gemini model generates: "Employees are entitled to 90 days of maternity leave (Source: HR_Policy.pdf, Page 4)."*

10. Response

The final, formatted answer — along with its source citations — travels back from the LLM through the backend to the frontend, where it is displayed to the user. It is also appended to the conversation history so that the user's next message can be understood as a contextual follow-up.

Example: *The chat window shows the maternity-leave answer plus a clickable "Sources" section listing HR_Policy.pdf, Page 4, ready for the user to ask a related follow-up question.*

Summary: Component Overview

The table below summarizes when each component executes and a representative technology choice, consolidating the detailed descriptions above into a single quick-reference view.

#	Component	Runs When	Example Technology
1	**User**	N/A (human)	Chat message via web or mobile UI
2	**Frontend**	Every query	Streamlit / React chat interface
3	**Backend**	Every query	FastAPI / Flask orchestration layer
4	**Document Processing**	Ingestion (offline)	pypdf, python-docx text extraction
5	**Chunking**	Ingestion (offline)	RecursiveCharacterTextSplitter
6	**Embedding Generation**	Ingestion (offline)	sentence-transformers (MiniLM)
7	**Vector Database**	Persisted after ingestion	ChromaDB / FAISS
8	**Retriever**	Every query	Cosine similarity top-k search
9	**LLM**	Every query	Gemini / GPT generation
10	**Response**	Every query	Formatted answer + citations